

OmniCart – Project Report

Artificial Intelligence Lab (AL-2002)

Group Members:

- Rayyan Aamir – 24K-0687 (Group Leader)
- Ahmed Shah Rashdi – 24K-0709
- Muhammad Usaid Khan – 24K-0832

Project Overview:

OmniCart is a full-stack AI-powered multi-vendor e-commerce platform built for digital retail market. The platform enables customers to browse and purchase electronics, vendors to manage their storefronts, and administrators to oversee the entire marketplace. The project integrates machine learning recommendation systems, automated workflow pipelines, a relational database backend, a RESTful API layer, and a modern reactive frontend.

Project Development Flow:

Development followed a database-first approach, with each layer built sequentially on top of the one below it.

Phase 1 - Database Design: The MySQL schema was designed first, establishing all eleven tables, relationships, constraints, and indexes before any application code was written. Stored procedures were authored alongside the schema to encapsulate critical transactional logic at the database level.

Phase 2 - Backend API: Flask routes and business logic were built against the confirmed schema. The authentication system, role-based access control, and input validation layer were established before feature-specific routes were added. The Python seed script was written to populate realistic test data.

Phase 3 - Machine Learning: With the database populated, the three ML pipelines were trained. The SVD model was trained first using the Amazon Reviews dataset, followed by the TF-IDF content engine built directly from OmniCart's product catalog. The sentiment pipeline was then applied to existing reviews and wired into the review submission endpoint. The hybrid recommendation function and LLaMA explanation integration were added last, connecting all ML components into a unified recommendations blueprint.

Phase 4 - Automation: n8n workflows were configured after the backend endpoints that feed them were finalized. Each workflow was connected to its corresponding Flask trigger endpoint and tested end-to-end with real SMTP delivery.

Phase 5 - Frontend: The React frontend was built last, consuming the fully functional API. Pages were developed role by role: public pages first, then customer, vendor, and admin views. Chart integrations and the ML-powered recommendation display were added once the corresponding API endpoints were confirmed stable.

Technology Stack:

Database: MySQL 8.0 with stored procedures, views, indexes, and constraints

Backend: Python 3.11, Flask, Flask-SQLAlchemy, Flask-JWT-Extended, Flask-Bcrypt, Flask-CORS, PyMySQL

Machine Learning: Scikit-learn (TF-IDF, cosine similarity), Surprise library (SVD collaborative filtering), TextBlob (sentiment analysis), NumPy, Pandas

AI Integration: NVIDIA NIM API (LLaMA 3.1 8B Instruct via OpenAI-compatible client) for natural language recommendation explanations

Automation: n8n (self-hosted workflow automation) with webhook triggers and SMTP email delivery

Frontend: React 19, Vite 8, Tailwind CSS 4, Framer Motion, Chart.js, React Router DOM 7, Axios, Lucide React

Deployment: Railway (backend and database), Vite dev server with proxy configuration

Database Layer:

Schema Design

The relational database consists of eleven tables: customers, vendors, admins, categories, products, orders, order_items, payments, reviews, cart, and recommendations. The schema enforces referential integrity through foreign keys with cascading behavior. For example, deleting a customer cascades to their orders, cart items, reviews, and recommendations. Products use a RESTRICT constraint on category deletion to prevent orphaned inventory. The categories table is self-referential, supporting parent-child hierarchy for nested classification.

Indexes are placed on high-frequency query columns: vendor_id and category_id on products, customer_id on orders and cart, order_id on order_items, and product_id on reviews. Unique constraints prevent duplicate cart entries per customer-product pair, duplicate reviews per customer-product pair, and duplicate recommendations per customer-product pair.

Check constraints enforce data quality at the database level: email format validation using LIKE pattern matching, phone number format validation using REGEXP for Pakistani mobile numbers (03XX-XXXXXXX), rating bounds between 1 and 5, and sentiment score bounds between 0.00 and 1.00.

Stored Procedures

Five stored procedures handle the most critical transactional operations atomically.

PlaceOrder accepts a customer ID and a JSON array of cart items. It validates the customer is active, iterates through each item to check stock availability and product status using row-level locking (FOR UPDATE) to prevent race conditions, calculates the total, inserts the order and all order items, decrements stock quantities, and clears the customer's cart. All of this happens within a single transaction with an EXIT HANDLER that rolls back on any SQL exception.

UpdateProductInventory validates vendor ownership before allowing stock or price modifications. It signals a custom SQLSTATE 45000 error if the product belongs to a different vendor, preventing cross-vendor manipulation.

DeactivateUser handles both customer and vendor deactivation. For customers it sets their pending orders to cancelled. For vendors it deactivates all their products simultaneously. Both actions commit in a single transaction.

ProcessPayment validates payment method from an allowed enum, checks the order belongs to the requesting customer, prevents duplicate payments, ensures the amount matches the order total, inserts the payment record, and updates the order status to confirmed.

CancelOrder reverses PlaceOrder by restoring stock quantities using a JOIN-based UPDATE, cancelling the order, and conditionally marking the payment as refunded or failed depending on its current state.

Complex SQL Queries

Five complex queries demonstrate advanced SQL features for the database course component.

Query 1 (Customer Order History) performs a four-table JOIN across customers, orders, order_items, and products to return a complete itemized purchase history with subtotals.

Query 2 (Monthly Revenue Report) uses DATE_FORMAT as a built-in function, SUM, COUNT, AVG, and GROUP BY to produce a monthly revenue breakdown for the last twelve months, excluding cancelled orders.

Query 3 (Top Customers by Spend) combines JOINS, aggregation, HAVING to filter by minimum order count, and a correlated subquery to include each customer's review count in a single result set.

Query 4 (Vendor Sales Performance) uses LEFT JOINS across vendors, products, order_items, and reviews with COALESCE to handle vendors who have no sales or no reviews, returning comprehensive sales summaries per vendor.

Query 5 (Category Sentiment Analysis) chains JOINS across categories, products, and reviews, uses AVG on both rating and sentiment_score, and includes a

correlated subquery that counts how many products in each category have an average sentiment score above 0.7.

Database Views

Three views are defined for reporting. `CustomerOrderHistory` joins customers, orders, `order_items`, and products into a flat denormalized view. `TopRatedProducts` joins products, reviews, and categories and applies a HAVING filter for average ratings above 3.5. `VendorSalesSummary` joins vendors, products, `order_items`, and reviews using LEFT JOINS to include vendors with zero sales.

Backend Layer:

Application Architecture

The Flask application uses the application factory pattern defined in `create_app()`. Extensions (SQLAlchemy, JWTManager, Bcrypt) are initialized separately in `extensions.py` and registered onto the app instance, avoiding circular imports. All models are imported inside the factory after `db.init_app()` to ensure the application context is established.

JWT authentication uses role-based claims embedded in the token payload. Tokens carry a role field (customer, vendor, or admin) and a name field alongside the subject identity. Admin tokens are given a one-year expiry while all other tokens expire in fifteen minutes. A custom `role_required` decorator sits on top of flask_jwt_extended's `jwt_required()` and reads the role claim from the token to enforce access control at the route level.

Route Organization

Routes are organized into eight blueprints: auth, customer, orders, vendor, admin, reviews, queries, and recommendations. Each blueprint is registered with a URL prefix.

The auth blueprint handles registration (customer and vendor), login for all three roles, profile retrieval, profile update, and password change. Vendor registration

does not return a token because vendors must await admin approval before they can log in.

The customer blueprint exposes public product listing with keyword search, category filtering, price range filtering, and safe whitelisted sorting. Cart operations (add, view, update quantity, remove item, clear cart) are protected by customer role requirement. The cart implementation handles orphaned items by silently deleting cart entries whose associated product has been deactivated or hard-deleted.

The orders blueprint delegates placement, cancellation, and payment processing to stored procedures, capturing custom SQLSTATE 45000 error messages from MySQL and surfacing them directly to the API consumer.

The admin blueprint provides platform-wide user management, vendor approval, category management with safe FK checks before deletion, and three n8n support endpoints that feed the automation workflows.

The reviews blueprint enforces verified purchase checking before allowing a review submission by querying `order_items` joined to orders filtered by confirmed, shipped, or delivered status. Sentiment scoring via `TextBlob` is computed synchronously on submission and stored in the `sentiment_score` column.

Validation

A shared validations module provides email format checking via the validators library, phone format checking via regex for Pakistani mobile numbers, date of birth parsing, string sanitization (converting empty strings to `None` before DB insertion), and stored procedure error extraction from MySQL exception strings using regex pattern matching on the double-quoted message text.

Machine Learning Components

This is the core AI contribution of the project. Three distinct ML systems are integrated.

SVD Collaborative Filtering (train_svd.py)

The recommendation engine uses matrix factorization to predict how much a given customer would enjoy a product they have not yet purchased or reviewed.

Dataset: Amazon product reviews CSV containing UserId, ProductId, and Score columns. The dataset contains millions of rows and is sampled to 200,000 records for training feasibility.

Preprocessing: Duplicate (UserId, ProductId) pairs are dropped keeping the most recent rating. Users with fewer than 3 ratings and products with fewer than 3 ratings are filtered out to remove noise and improve generalization.

Model: The Surprise library's SVD implementation uses 50 latent factors and 20 training epochs. Three-fold cross-validation is performed before fitting the final model on the full training set. Cross-validation results show RMSE and MAE values that indicate reasonable rating prediction accuracy for implicit preference modeling.

Integration: During inference, each of the platform's active product IDs is mapped to an Amazon product ID using a constructed index (B followed by zero-padded integer). The trained model predicts a rating for the current customer-product pair and returns the top N products by predicted score.

Limitation acknowledged: Because OmniCart products do not exist in the Amazon training data, the SVD predictions serve as a proxy for general preference tendencies rather than precise product-level predictions. The hybrid system compensates for this.

TF-IDF Content-Based Filtering (train_tfidf.py)

A content-based engine finds products similar to a given product by analyzing textual features.

Feature Engineering: For each active product in OmniCart's database, a content string is constructed by concatenating the product name (duplicated for higher

weighting), brand (duplicated), category name, parent category name, and description. This string is lowercased and stripped.

Vectorization: Scikit-learn's TfidfVectorizer processes the content strings with a vocabulary limit of 5,000 features, English stop word removal, and a unigram-bigram n-gram range. The resulting sparse matrix captures term importance across the product catalog.

Similarity: Cosine similarity is computed pairwise across the TF-IDF matrix, producing a square matrix where each entry represents the textual similarity between two products. This is stored in the pickle alongside the vectorizer and product-to-index mappings.

Integration: The `get_similar_products()` function takes a product ID, looks up its index, sorts all other products by cosine similarity score descending, and returns the top N results. This powers the "You May Also Like" section on product detail pages.

Hybrid Recommendation System

The production recommendation function `get_hybrid_recommendations()` combines both engines.

SVD scores are normalized to a 0–1 range using min-max scaling across all products for the current user. TF-IDF similarity scores from a recently viewed product (if provided) are used as the content signal. The final score is computed as $0.6 \times \text{SVD score} + 0.4 \times \text{TF-IDF score}$ when a recently viewed product is available, and purely SVD-based otherwise. Products are ranked by this combined score and the top N are returned.

Recommendations are stored in the recommendations table with their score and an explanation string. The stored recommendations are served on subsequent requests without re-running the ML pipeline, making the customer dashboard fast.

LLaMA-Based Recommendation Explanations

When a customer requests an explanation for a specific recommendation, the `/api/recommendations/explain` endpoint calls the NVIDIA NIM API using the OpenAI-compatible client pointed at the NVIDIA inference endpoint. The model used is `meta/llama-3.1-8b-instruct`. A system prompt establishes the persona as a friendly shopping assistant for OmniCart, and the user prompt requests a 1–2 sentence explanation of why a specific product is a great fit for a named customer in a named category. The generated explanation is stored back to the database in the recommendation's explanation field.

Sentiment Analysis (`train_sentiment.py`)

Review sentiment is computed using TextBlob's `sentiment.polarity` attribute, which returns a float between -1.0 and 1.0. This is normalized to a 0.0–1.0 range using the formula $(\text{polarity} + 1) / 2$. The pipeline is validated against 5,000 rows of Amazon reviews by comparing TextBlob's predicted label (positive above 0.6, negative below 0.4, neutral otherwise) against the actual star rating (positive for 4–5, negative for 1–2, neutral for 3). Accuracy is printed to confirm the analyzer is performing reasonably before the model artifact is saved.

The trained pipeline is then applied to all existing OmniCart reviews in the database, updating each review's `sentiment_score`. On new review submission through the API, sentiment is computed in real time before the review is committed.

The Category Sentiment Analysis query (Query 5) aggregates these scores to show average sentiment per category alongside average rating, giving administrators insight into which product categories generate the most positive customer response.

AI-Generated Product Descriptions:

Beyond recommendation explanations, LLaMA 3.1 is also used to automatically generate product descriptions for vendors when they are adding a new product to the catalog.

When a vendor fills in the product name, category, price, and brand on the Add Product page and clicks the "Sync AI Generation" button, the frontend sends

those fields to a backend endpoint which constructs a prompt for the NVIDIA NIM API. The system prompt instructs the model to act as an e-commerce copywriter and produce a compelling product description in two sentences maximum. The user prompt passes in the product name, brand, category name, and price in Pakistani Rupees.

The generated description is returned to the frontend and populated into the description text area, where the vendor can accept it as-is or edit it before submitting. If the API call fails for any reason, the frontend displays a toast notification telling the vendor to write the description manually, ensuring the feature degrades gracefully without blocking product creation.

This removes a significant friction point for vendors who may not be experienced writers, ensuring product listings are consistently descriptive and professionally worded across the catalog.

n8n Automation Workflows

Four n8n automation workflows handle event-driven communication and are connected to the Flask backend via webhook triggers.

Order Confirmation (order-confirmation.json)

This workflow is triggered by a POST webhook at the path order-placed. The Flask backend fires this webhook non-blocking (using a try-except with timeout=3 seconds) immediately after a successful PlaceOrder stored procedure call. The webhook body contains the order ID, customer ID, total amount, and an items array with product names, quantities, and unit prices. The n8n workflow sends an HTML-formatted email confirmation to the customer listing all ordered items with individual subtotals and the grand total.

Cart Abandonment Reminder (cart_abandonment_reminder.json)

This workflow receives a POST to trigger-abandoned-carts. The Flask backend queries cart items older than 24 hours joined to active customers and their associated products. The payload groups items by customer into a structured list of customer objects each containing their cart contents. The n8n workflow first evaluates an If node to ensure count is greater than zero, then uses a Code node

to spread the customers array into individual items, feeds them into a SplitInBatches loop, and sends each customer a personalized HTML email listing their abandoned products with quantities and prices. The workflow can be triggered manually by an admin through the admin dashboard or scheduled.

Low Stock Alert (low_stock_alert.json)

This workflow receives a POST to trigger-low-stock. The backend queries products with stock_qty below a configurable threshold (defaulting to 10), joining to the vendor table to include the vendor email and store name. The n8n workflow checks count is positive, then loops through each low-stock product and sends a warning email to the vendor's registered email address identifying the specific product and current stock level.

Weekly Deals Digest (weekly_deals_digest.json)

This workflow receives a POST to trigger-weekly-digest. The backend fetches all active customers and for each one retrieves their top 5 stored recommendations ordered by score descending. The payload contains customer names, emails, and recommendation arrays with product names, prices, and AI-generated explanations. The n8n workflow filters for non-empty customer lists, maps them into individual items, and sends each customer a personalized HTML email formatted as a numbered list of recommended products with the LLaMA-generated explanation for each.

All three trigger workflows support both scheduled execution within n8n and on-demand triggering from the admin panel, giving administrators full control over when campaigns run.

Frontend Layer

The frontend is an application built with React 19 and Vite 8, styled with Tailwind CSS 4 and animated with Framer Motion 12.

Architecture

The application uses React Context for shared state: AuthContext manages JWT token storage and user identity, CartContext maintains cart state synchronized with the backend, and ThemeContext persists dark mode preference to localStorage. Routing is handled by React Router DOM 7 with nested routes under a Layout component that renders the persistent Navbar and Footer.

A custom Axios instance in api/axios.js attaches the JWT Bearer token to every request via a request interceptor and handles 401 responses globally by clearing stored credentials and redirecting to the login page.

Pages and Features

Public pages include Home (hero section, category showcase, trending products from the ML endpoint, features section), Products (searchable, filterable, sortable catalog with pagination), and ProductDetail (with similar products from TF-IDF, review submission, and average rating display).

Customer-protected pages include Cart, Checkout (with payment method selection), CustomerDashboard (with spending chart, interests doughnut chart, recent orders, and ML recommendations), Orders list, and OrderDetail with a step progress visualization.

Vendor-protected pages include VendorDashboard (sales chart, inventory alerts, quick product list), MyProducts (table with edit and deactivate actions), AddProduct (with AI description generation via LLaMA), and VendorOrders.

The AdminDashboard provides platform statistics, monthly revenue and category charts, customer management with toggle activation, vendor approval, and order status management.

UI Components

The BorderGlow component is a custom CSS-based interactive card that tracks mouse proximity and cursor angle using JavaScript event listeners, applying CSS custom properties to drive conic gradient masks for an edge-lighting glow effect. AnalyticsChart wraps Chart.js with theme-aware configuration. ProductCard

uses Framer Motion for entrance animations and includes hover-revealed action buttons.

Member Contributions:

Ahmed Shah Rashdi led the machine learning pipeline design and implementation. Trained the SVD collaborative filtering model using the Amazon Reviews dataset including preprocessing, cross-validation, and model serialization. Built the TF-IDF content-based engine including feature engineering, vectorization, cosine similarity computation, and integration with Flask. Implemented the hybrid recommendation scoring function. Integrated the NVIDIA NIM API for LLaMA-based recommendation explanations. Designed and implemented the sentiment analysis pipeline using TextBlob including Amazon validation and database update script. Implemented the recommendations Flask blueprint and all associated API endpoints. Designed and built the complete React frontend including all pages, context providers, the custom Axios instance with interceptors, and the routing structure. Implemented the BorderGlow interactive UI component and the AnalyticsChart wrapper. Built the admin dashboard with charts and management tables. Configured the Vite proxy for local development.

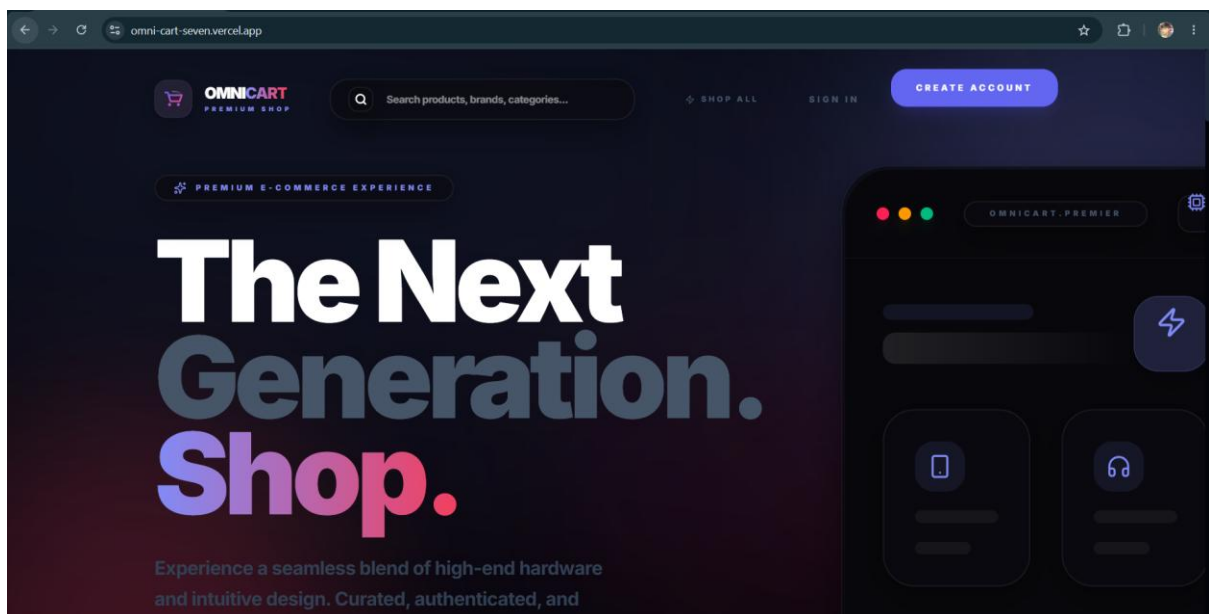
Muhammad Usaid Khan designed the complete MySQL database schema including all eleven tables, foreign key relationships, cascade behaviors, check constraints, and indexes. Authored all five stored procedures (PlaceOrder, UpdateProductInventory, DeactivateUser, ProcessPayment, CancelOrder) with full transaction management and custom error signaling. Wrote the five complex SQL queries for the database course evaluation. Defined the three database views. Wrote the SQL seed file for standalone database demonstration.

Rayyan Aamir built the Flask backend including the application factory, extensions setup, all eight route blueprints, JWT authentication with role-based access control, and the custom role_required middleware decorator. Implemented input validation, sanitization, and stored procedure error extraction. Wrote the Python seed script with idempotent get_or_create logic. Designed and

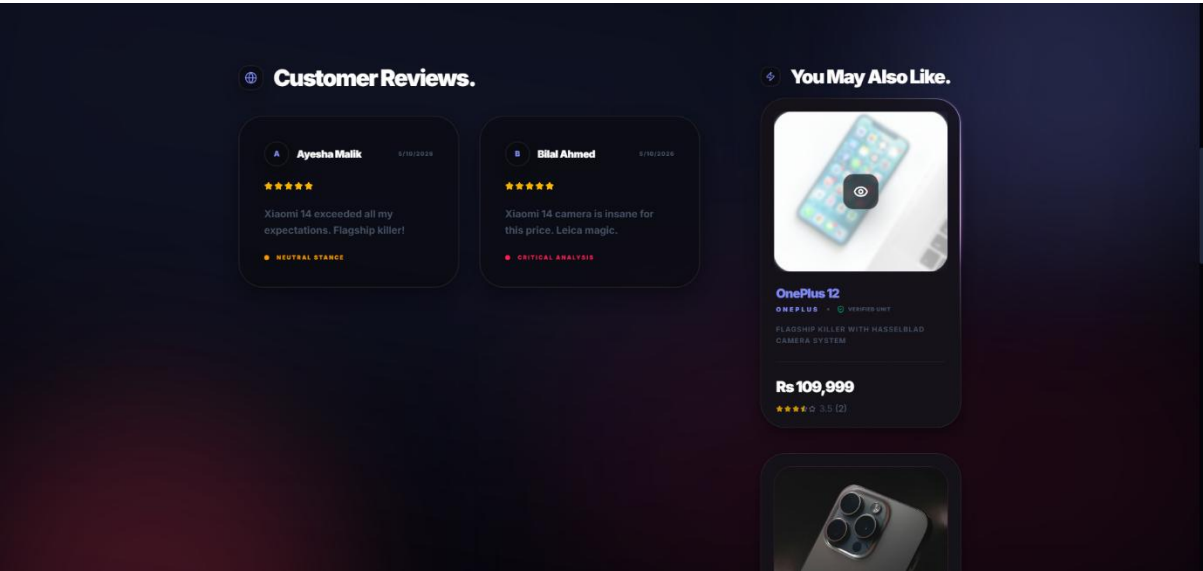
implemented all four n8n automation workflows. Configured webhook triggers and connected them to the Flask backend endpoints. Set up SMTP credentials and authored the HTML email templates for order confirmation, cart abandonment, low stock alerts, and the weekly personalized digest. Tested end-to-end workflow execution. Configured the Railway deployment and managed environment variables. Led overall system architecture and integration between all layers including the ML layer, API layer, automation webhooks, and frontend API consumption.

Screenshots:

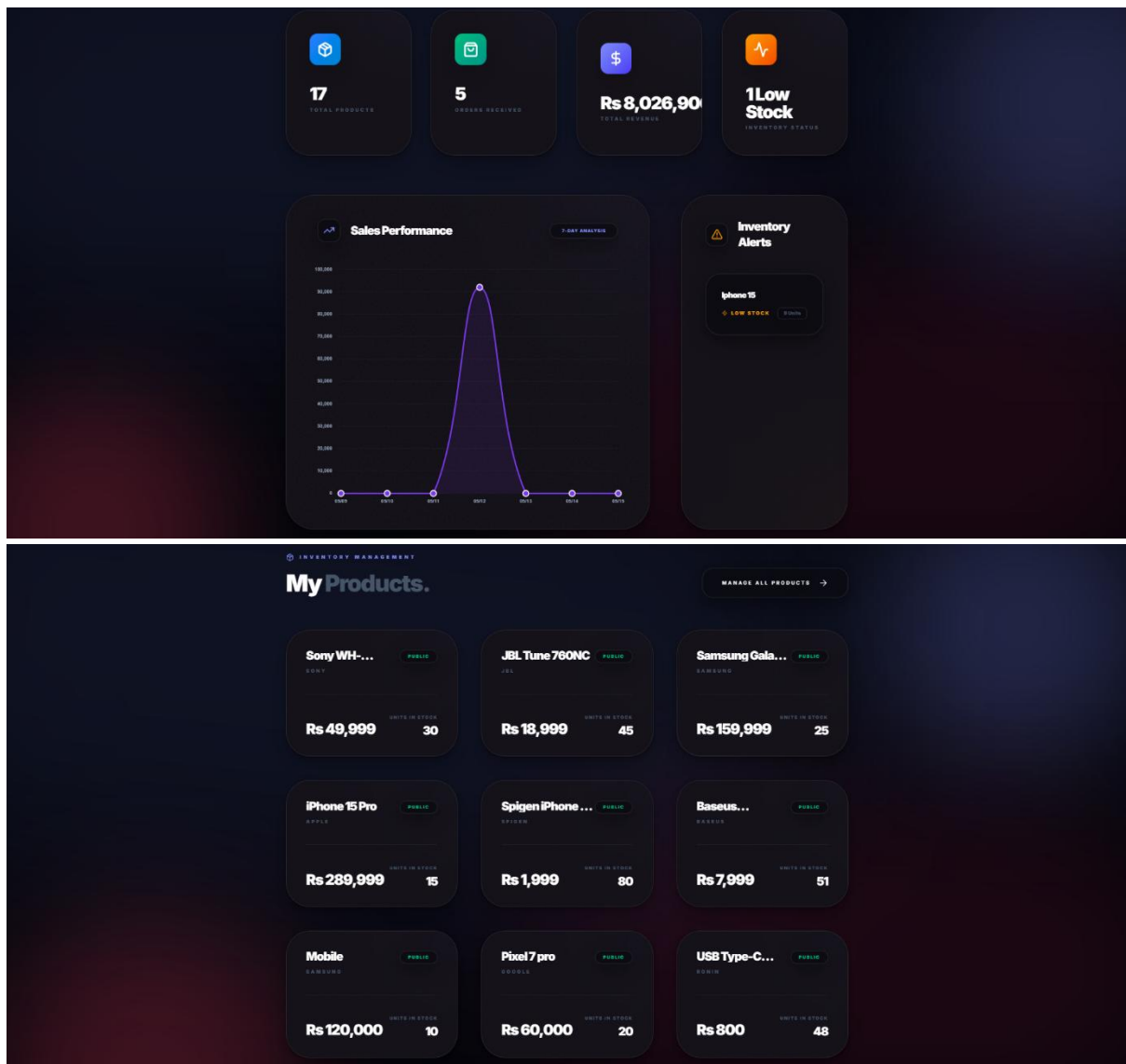
Homepage



Product View and Recommender System in Action:



Vendor Dashboard:



AI-Generated Product Description:

DEPLOYMENT VALUE (PKR) *

250000

INVENTORY ALLOCATION

34

✓ DESCRIPTION GENERATED BY AI!

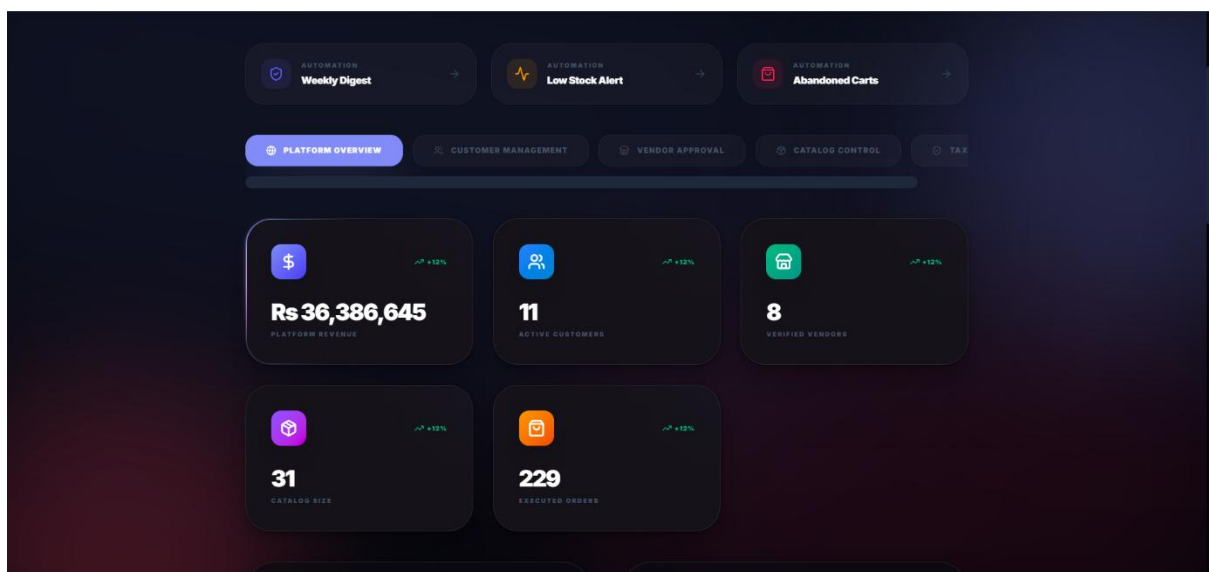
TECHNICAL SPECIFICATION

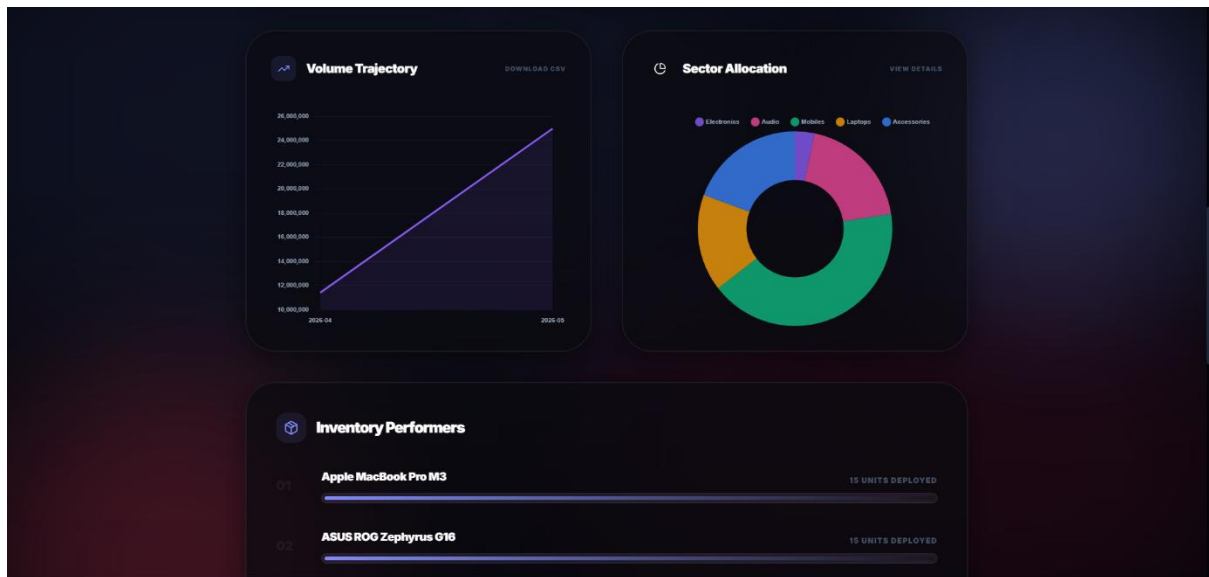
SYNC AI GENERATION

"Unlock unparalleled mobile luxury with the Samsung S26 Ultra, boasting an exceptional 6.8" Dynamic AMOLED display, powerful Exynos 2100 chipset, and a long-lasting 5000mAh battery. Experience cutting-edge photography with a 108MP primary sensor, stunning 8K video recording, and seamless performance in this premium flagship device."

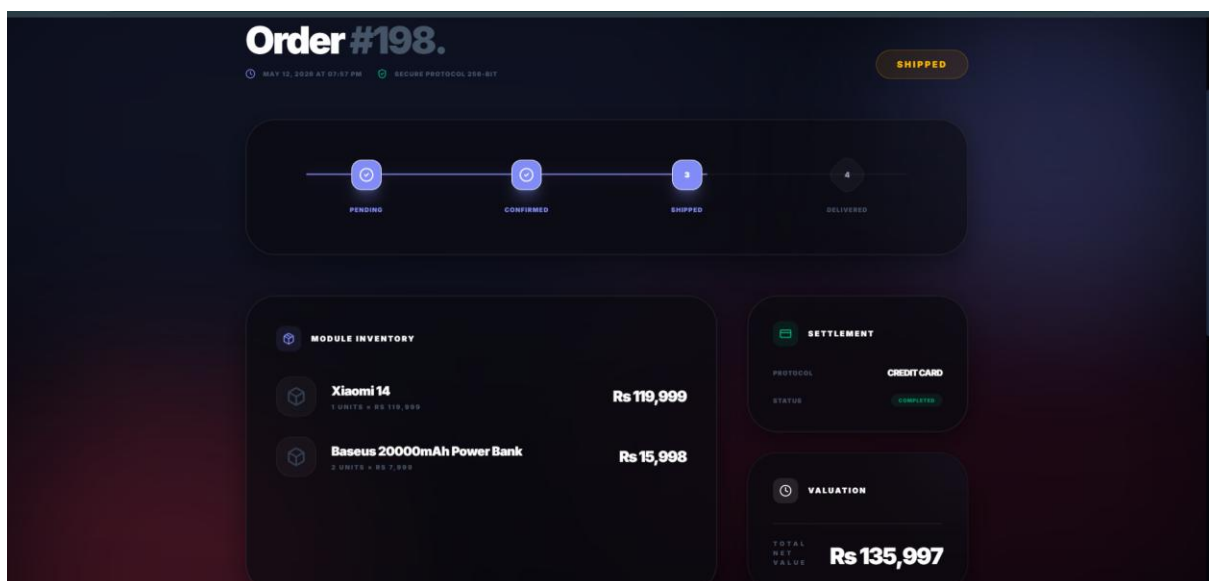
INITIALIZE DEPLOYMENT

Admin Dashboard:

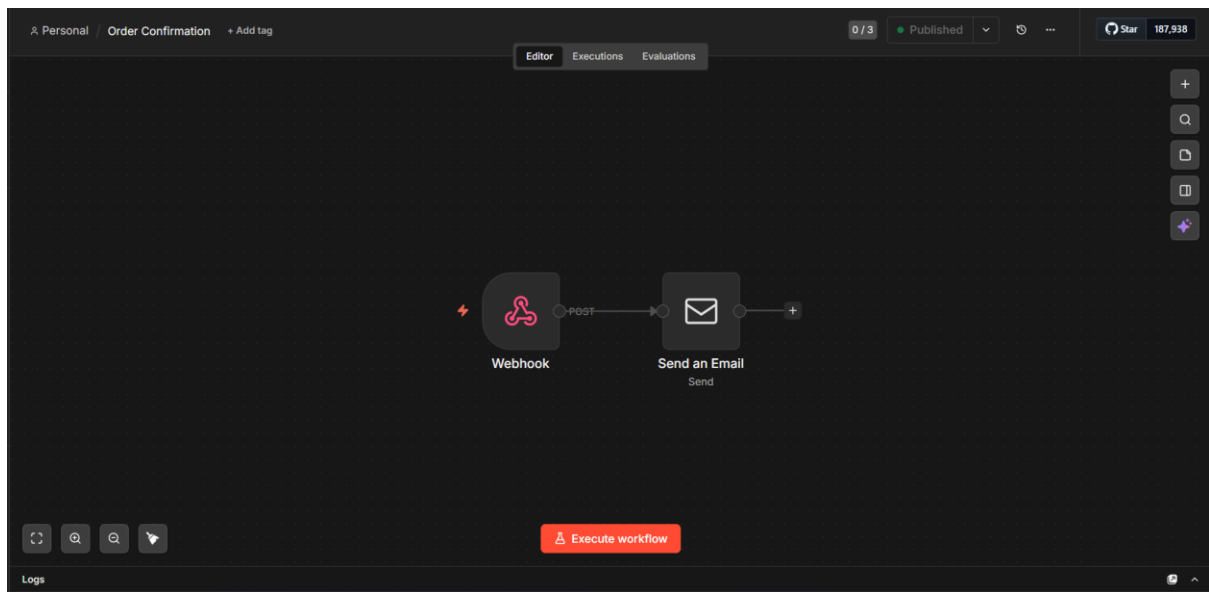




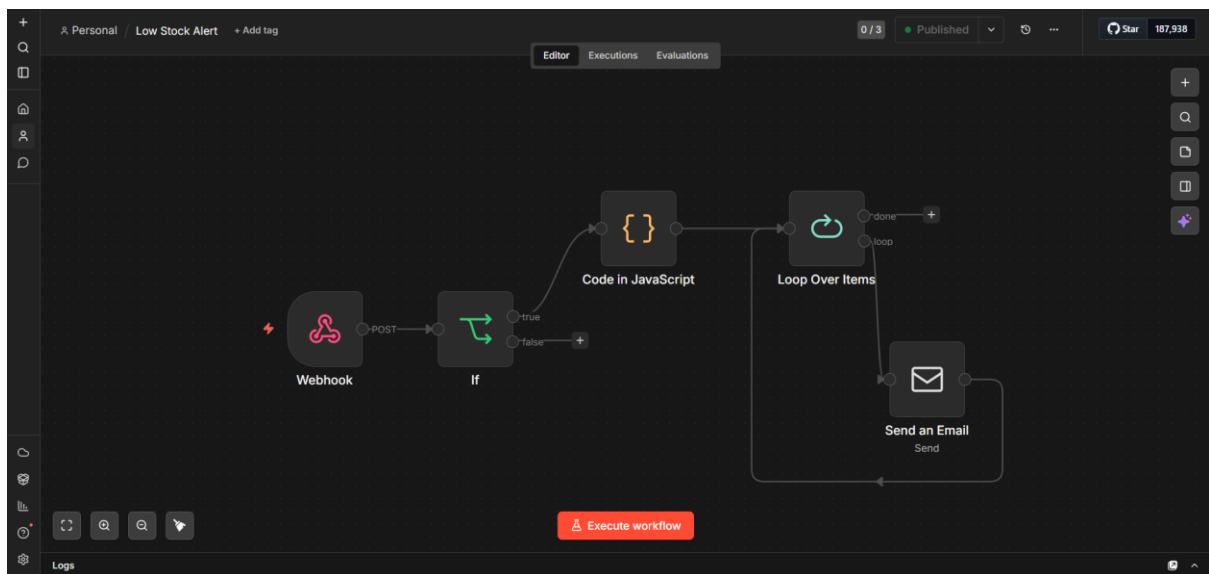
Customer Order Tracking:



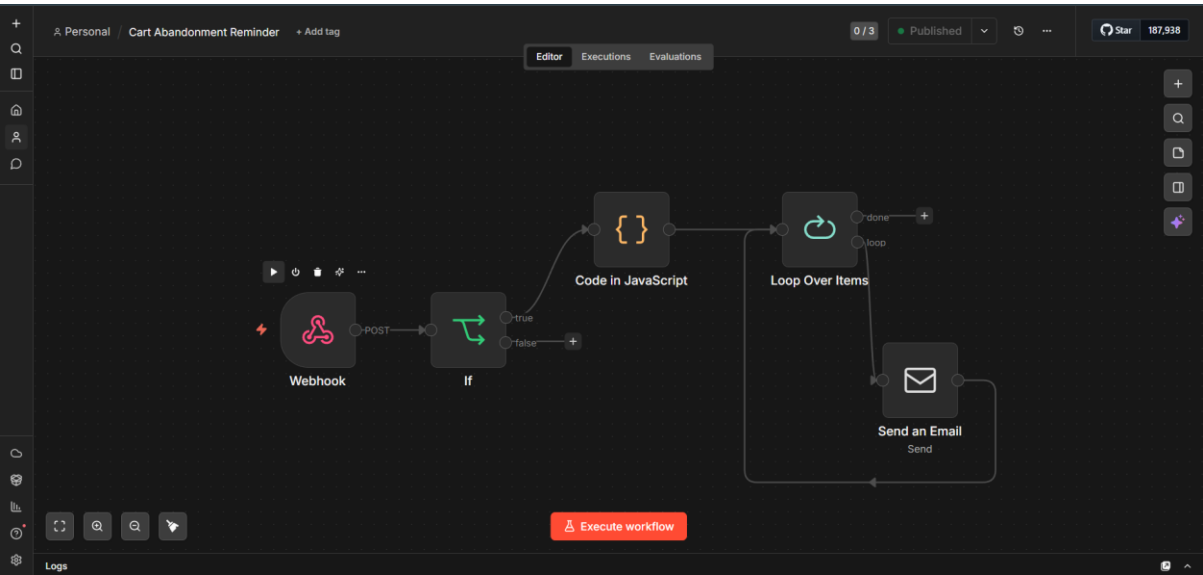
Order Confirmation Workflow:



Low Stock Workflow:



Cart Abandonment Workflow:



Weekly Deals Digest Workflow:

